

Projectquestion:

Models:

Skill.cs

using System.ComponentModel.DataAnnotations;

```
namespace SmartSkillExchangeApp.Models
{
    public class Skill
    {
        public int Id { get; set; }

        // skill title
        [Required]
        [StringLength(100)]
        public string Title { get; set; } = string.Empty;

        // skill description
        [Required]
        public string Description { get; set; } = string.Empty;

        // foreign key user
        public int UserId { get; set; }

        // navigation property
        public User? User { get; set; }

        public List<Booking>? Bookings { get; set; }
        public List<Review>? Reviews { get; set; }
    }
}
```

}User.cs

using System.ComponentModel.DataAnnotations;

```
namespace SmartSkillExchangeApp.Models
{
    public class User
    {
        public int Id { get; set; }

        [Required]
        [StringLength(50)]
        public string Name { get; set; } = string.Empty;

        [Required]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        public List<Skill>? Skills { get; set; }
        public List<Booking>? Bookings { get; set; }

        public Wallet? Wallet { get; set; }
    }
}
```

}Booking.cs

using System.ComponentModel.DataAnnotations;

```
namespace SmartSkillExchangeApp.Models
{
    public class Booking
    {

```

```

        public int Id { get; set; }

        [Required]
        public DateTime Date { get; set; }

        public int UserId { get; set; }
        public int SkillId { get; set; }

        public User? User { get; set; }
        public Skill? Skill { get; set; }
    }
}Review.cs
using System.ComponentModel.DataAnnotations;

namespace SmartSkillExchangeApp.Models
{
    public class Review
    {
        public int Id { get; set; }

        [Required]
        public string Comment { get; set; } = string.Empty;

        public int Rating { get; set; }

        public int SkillId { get; set; }

        public Skill? Skill { get; set; }
    }
}Wallet.cs
namespace SmartSkillExchangeApp.Models
{
    public class Wallet
    {
        public int Id { get; set; }

        public int Points { get; set; } = 100;

        public int UserId { get; set; }

        public User? User { get; set; }
    }
}Data/ApplicationDbContext.cs
using Microsoft.EntityFrameworkCore;
using SmartSkillExchangeApp.Models;

namespace SmartSkillExchangeApp.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<User> Users { get; set; }
        public DbSet<Skill> Skills { get; set; }
    }
}

```

```

        public DbSet<Booking> Bookings { get; set; }
        public DbSet<Review> Reviews { get; set; }
        public DbSet<Wallet> Wallets { get; set; }
    }
}
Controller:
SkillController:
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SmartSkillExchangeApp.Data;
using SmartSkillExchangeApp.Models;

namespace SmartSkillExchangeApp.Controllers
{
    public class SkillController : Controller
    {
        private readonly ApplicationDbContext _context;

        public SkillController(ApplicationDbContext context)
        {
            _context = context;
        }

        // show all skills
        public async Task<IActionResult> Index(string search)
        {
            var skills = _context.Skills
                .Include(s => s.User)
                .AsQueryable();

            // search using LINQ
            if (!string.IsNullOrEmpty(search))
            {
                skills = skills.Where(s => s.Title.Contains(search));
            }

            return View(await skills.ToListAsync());
        }

        // create form
        public IActionResult Create()
        {
            return View();
        }

        // save skill
        [HttpPost]
        public async Task<IActionResult> Create(Skill skill)
        {
            if (ModelState.IsValid)
            {
                var userId = HttpContext.Session.GetInt32("UserId");

                if (userId == null)
                {
                    TempData["Error"] = "Login first";
                    return RedirectToAction("Login", "Account");
                }
            }
        }
    }
}

```

```

        skill.UserId = userId.Value;

        _context.Add(skill);
        await _context.SaveChangesAsync();

        TempData["Success"] = "Skill added";

        return RedirectToAction("Index");
    }

    return View(skill);
}

// edit form
public async Task<ActionResult> Edit(int id)
{
    var skill = await _context.Skills.FindAsync(id);
    if (skill == null) return NotFound();

    return View(skill);
}

// update skill

[HttpPost]
public async Task<ActionResult> Edit(Skill skill)
{
    if (ModelState.IsValid)
    {
        var existingSkill = await _context.Skills.FindAsync(skill.Id);

        if (existingSkill == null) return NotFound();

        // update only fields
        existingSkill.Title = skill.Title;
        existingSkill.Description = skill.Description;

        await _context.SaveChangesAsync();

        TempData["Success"] = "Updated";

        return RedirectToAction("Index");
    }

    return View(skill);
}

// delete skill
public async Task<ActionResult> Delete(int id)
{
    var skill = await _context.Skills.FindAsync(id);

    if (skill != null)
    {
        _context.Remove(skill);
        await _context.SaveChangesAsync();
    }
}

```

```

        return RedirectToAction("Index");
    }

    // details page
    public async Task<ActionResult> Details(int id)
    {
        var skill = await _context.Skills
            .Include(s => s.User)
            .Include(s => s.Reviews)
            .FirstOrDefaultAsync(s => s.Id == id);

        if (skill == null) return NotFound();

        return View(skill);
    }
}

AccountController.cs
using Microsoft.AspNetCore.Mvc;
using SmartSkillExchangeApp.Data;
using SmartSkillExchangeApp.Models;

namespace SmartSkillExchangeApp.Controllers
{
    public class AccountController : Controller
    {
        private readonly ApplicationDbContext _context;

        public AccountController(ApplicationDbContext context)
        {
            _context = context;
        }

        // register form
        public IActionResult Register()
        {
            return View();
        }

        // save user
        [HttpPost]
        public IActionResult Register(User user)
        {
            if (ModelState.IsValid)
            {
                _context.Users.Add(user);
                _context.SaveChanges();

                // create wallet
                var wallet = new Wallet
                {
                    UserId = user.Id,
                    Points = 100
                };

                _context.Wallets.Add(wallet);
                _context.SaveChanges();
            }
        }
    }
}

```

```

        TempData["Success"] = "Registered";

        return RedirectToAction("Login");
    }

    return View(user);
}

// login form
public IActionResult Login()
{
    return View();
}

// login user
[HttpPost]
public IActionResult Login(string email)
{
    var user = _context.Users.FirstOrDefault(u => u.Email == email);

    if (user != null)
    {
        HttpContext.Session.SetInt32("UserId", user.Id);

        TempData["Success"] = "Logged in";

        return RedirectToAction("Index", "Skill");
    }

    TempData["Error"] = "Invalid email";
    return View();
}

// logout
public IActionResult Logout()
{
    HttpContext.Session.Clear();
    return RedirectToAction("Login");
}
}

BookingController.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SmartSkillExchangeApp.Data;
using SmartSkillExchangeApp.Models;

namespace SmartSkillExchangeApp.Controllers
{
    public class BookingController : Controller
    {
        private readonly ApplicationDbContext _context;

        public BookingController(ApplicationDbContext context)
        {
            _context = context;
        }
    }
}

```

```

// show bookings
public async Task<IActionResult> Index()
{
    var bookings = await _context.Bookings
        .Include(b => b.Skill)
        .Include(b => b.User)
        .ToListAsync();

    return View(bookings);
}

// create booking form
public IActionResult Create(int skillId)
{
    ViewBag.SkillId = skillId;
    return View();
}

// save booking
[HttpPost]
public async Task<IActionResult> Create(Booking booking)
{
    var userId = HttpContext.Session.GetInt32("UserId");

    if (userId == null)
    {
        TempData["Error"] = "Login first";
        return RedirectToAction("Login", "Account");
    }

    booking.UserId = userId.Value;

    // wallet logic
    var wallet = await _context.Wallets
        .FirstOrDefaultAsync(w => w.UserId == userId);

    if (wallet == null || wallet.Points < 10)
    {
        TempData["Error"] = "Not enough points";
        return RedirectToAction("Index", "Skill");
    }

    wallet.Points -= 10;

    _context.Add(booking);
    await _context.SaveChangesAsync();

    TempData["Success"] = "Booking done";

    return RedirectToAction("Index");
}

// delete booking
public async Task<IActionResult> Delete(int id)
{
    var booking = await _context.Bookings.FindAsync(id);

    if (booking != null)

```

```

        {
            _context.Remove(booking);
            await _context.SaveChangesAsync();
        }

        return RedirectToAction("Index");
    }
}
}
}
Program.cs
using Microsoft.EntityFrameworkCore;
using SmartSkillExchangeApp.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddSession();

var app = builder.Build();

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseSession();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Skill}/{action=Index}/{id?}");

app.Run();
In .csproj
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="8.0.5" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.5" />
  </ItemGroup>

</Project>
Views
Skill/Create.cshtml
@model SmartSkillExchangeApp.Models.Skill

<h2>Create Skill</h2>

```

```
<form asp-action="Create" method="post">
  <label>Title</label>
  <input asp-for="Title" />
  <span asp-validation-for="Title"></span>

  <br />

  <label>Description</label>
  <input asp-for="Description" />

  <br />

  <button type="submit">Save</button>
</form>
Index.cshtml
@model List<SmartSkillExchangeApp.Models.Skill>
```

```
<h2>Skills</h2>
```

```
<a href="/Skill/Create">Add Skill</a>
```

```
@foreach (var skill in Model)
{
  <p>
    @skill.Title

    <a href="/Skill/Details/@skill.Id">Details</a> |
    <a href="/Skill/Edit/@skill.Id">Edit</a> |
    <a href="/Skill/Delete/@skill.Id">Delete</a>
  </p>
}
Edit.cshtml
@model SmartSkillExchangeApp.Models.Skill
```

```
<h2>Edit Skill</h2>
```

```
<form asp-action="Edit" method="post">

  <input type="hidden" asp-for="Id" />

  <label>Title</label>
  <input asp-for="Title" />

  <br />

  <label>Description</label>
  <input asp-for="Description" />

  <br />

  <button type="submit">Update</button>
</form>
Details.cshtml
@model SmartSkillExchangeApp.Models.Skill
```

```
<h2>Skill Details</h2>
```

```
<p><strong>Title:</strong> @Model.Title</p>
<p><strong>Description:</strong> @Model.Description</p>
```

```
<a href="/Skill/Edit/@Model.Id">Edit</a> |
<a href="/Skill/Delete/@Model.Id">Delete</a> |
<a href="/Skill">Back</a>Delete.cs
@model SkillExchangeApp.Models.Skill
```

```
<h2>Delete Skill</h2>
```

```
<!-- Requirement 9 CRUD Delete confirmation -->
<p>Are you sure you want to delete this skill?</p>
```

```
<!-- Requirement 9 Display data before delete -->
<p><strong>Title:</strong> @Model.Title</p>
<p><strong>Description:</strong> @Model.Description</p>
```

```
<!-- Requirement 6 Form submission -->
<form asp-action="Delete" method="post">
```

```
    <!-- Requirement 9 Identify record to delete -->
    <input type="hidden" asp-for="Id" />
```

```
    <!-- Requirement 9 Delete operation -->
    <button type="submit">Delete</button>
```

```
    <!-- Requirement 5 Navigation -->
    <a href="/Skill">Cancel</a>
```

```
</form>
Booking/
Create.cshtml
@model SmartSkillExchangeApp.Models.Booking
```

```
<h2>Book Skill</h2>
```

```
<form asp-action="Create" method="post">
```

```
    <input type="hidden" asp-for="SkillId" value="@ViewBag.SkillId" />
```

```
    <label>Date</label>
    <input asp-for="Date" type="date" />
```

```
    <br />
```

```
    <button type="submit">Book</button>
```

```
</form>
Index.cshtml
@model List<SmartSkillExchangeApp.Models.Booking>
```

```
<h2>Bookings</h2>
```

```
@foreach (var booking in Model)
{
    <p>@booking.Id - @booking.Date</p>
}
```

```
Account/
Register.cshtml
@model SmartSkillExchangeApp.Models.User
```

```
<h2>Register</h2>
```

```
<form asp-action="Register" method="post">
```

```
    <label>Name</label>
```

```
    <input asp-for="Name" />
```

```
    <br />
```

```
    <label>Email</label>
```

```
    <input asp-for="Email" />
```

```
    <br />
```

```
    <button type="submit">Register</button>
```

```
</form>
```

Login.cshtml

```
<h2>Login</h2>
```

```
<form asp-action="Login" method="post">
```

```
    <label>Email</label>
```

```
    <input name="email" />
```

```
    <br />
```

```
    <button type="submit">Login</button>
```

```
</form>
```

Order Management API

Model/Order.cs

```
namespace OrderApi.Models
```

```
{
```

```
    public class Order
```

```
    {
```

```
        public int Id { get; set; }
```

```
        public string CustomerName { get; set; } = string.Empty;
```

```
        public double TotalAmount { get; set; }
```

```
        public string Status { get; set; } = string.Empty;
```

```
    }
```

```
}
```

DTO/OrderDto.cs

```
namespace OrderApi.DTOs
```

```
{
```

```
    public class OrderDto
```

```
    {
```

```
        public string CustomerName { get; set; } = string.Empty;
```

```
        public double TotalAmount { get; set; }
```

```
    }
```

```
}
```

DbContext/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
```

```
using OrderApi.Models;
```

```
namespace OrderApi.Data
```

```
{
```

```
    public class AppDbContext : DbContext
```

```

    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Order> Orders => Set<Order>();
    }
}
Repository
using OrderApi.Models;

```

```

namespace OrderApi.Repositories
{
    public interface IOrderRepository
    {
        Task<IEnumerable<Order>> GetAllOrders();
        Task<Order?> GetOrderByld(int id);
        Task<Order> CreateOrder(Order order);
        Task UpdateStatus(int id, string status);
        Task DeleteOrder(int id);
    }
}

```

```

OrderRepository.cs
using Microsoft.EntityFrameworkCore;
using OrderApi.Data;
using OrderApi.Models;

```

```

namespace OrderApi.Repositories
{
    public class OrderRepository : IOrderRepository
    {
        private readonly AppDbContext _context;

        public OrderRepository(AppDbContext context)
        {
            _context = context;
        }

        public async Task<IEnumerable<Order>> GetAllOrders()
        {
            return await _context.Orders.ToListAsync();
        }

        public async Task<Order?> GetOrderByld(int id)
        {
            return await _context.Orders.FindAsync(id);
        }

        public async Task<Order> CreateOrder(Order order)
        {
            if (order.TotalAmount > 50000)
                order.Status = "High Value Order";
            else
                order.Status = "Pending";

            _context.Orders.Add(order);
            await _context.SaveChangesAsync();
        }
    }
}

```

```

        return order;
    }

    public async Task UpdateStatus(int id, string status)
    {
        var order = await _context.Orders.FindAsync(id);

        if (order != null)
        {
            order.Status = status;
            await _context.SaveChangesAsync();
        }
    }

    public async Task DeleteOrder(int id)
    {
        var order = await _context.Orders.FindAsync(id);

        if (order != null)
        {
            _context.Orders.Remove(order);
            await _context.SaveChangesAsync();
        }
    }
}

```

```

 AutoMapper
 MappingProfile.cs
 using AutoMapper;
 using OrderApi.DTOs;
 using OrderApi.Models;

```

```

 namespace OrderApi.Mapping
 {
     public class MappingProfile : Profile
     {
         public MappingProfile()
         {
             CreateMap<OrderDto, Order>();
         }
     }
 }

```

```

 Controller
 OrderController.cs
 using Microsoft.AspNetCore.Mvc;
 using OrderApi.DTOs;
 using OrderApi.Models;
 using OrderApi.Repositories;
 using AutoMapper;

```

```

 namespace OrderApi.Controllers
 {
     [ApiController]
     [Route("api/orders")]
     public class OrderController : ControllerBase
     {
         private readonly IOrderRepository _repo;
         private readonly IMapper _mapper;
     }
 }

```

```

public OrderController(IOrderRepository repo, IMapper mapper)
{
    _repo = repo;
    _mapper = mapper;
}

[HttpGet]
public async Task<ActionResult> GetAll()
{
    var data = await _repo.GetAllOrders();
    return Ok(data);
}

[HttpGet("{id}")]
public async Task<ActionResult> Get(int id)
{
    var order = await _repo.GetOrderById(id);
    if (order == null)
        return NotFound();
    return Ok(order);
}

[HttpPost]
public async Task<ActionResult> Create(OrderDto dto)
{
    if (string.IsNullOrEmpty(dto.CustomerName) || dto.TotalAmount <= 0)
        return BadRequest("Invalid input");

    var order = _mapper.Map<Order>(dto);
    var result = await _repo.CreateOrder(order);

    return CreatedAtAction(nameof(Get), new { id = result.Id }, result);
}

[HttpPut("{id}")]
public async Task<ActionResult> UpdateStatus(int id, string status)
{
    if (status != "Pending" && status != "Completed" && status != "Cancelled")
        return BadRequest("Invalid status");

    await _repo.UpdateStatus(id, status);
    return Ok();
}

[HttpDelete("{id}")]
public async Task<ActionResult> Delete(int id)
{
    await _repo.DeleteOrder(id);
    return NoContent();
}
}

Middleware
ExceptionMiddleware.cs
namespace OrderApi.Middleware
{
    public class ExceptionMiddleware

```

```
{
    private readonly RequestDelegate _next;

    public ExceptionMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task Invoke(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch
        {
            context.Response.StatusCode = 500;
            context.Response.ContentType = "application/json";

            await context.Response.WriteAsync(
                "{\"error\": \"Something went wrong\"}");
        }
    }
}

}

BackgroundService
OrderBackgroundService.cs
using Microsoft.Extensions.Hosting;

namespace OrderApi.Services
{
    public class OrderBackgroundService : BackgroundService
    {
        protected override async Task ExecuteAsync(CancellationToken stoppingToken)
        {
            while (!stoppingToken.IsCancellationRequested)
            {
                Console.WriteLine("Checking pending orders...");
                await Task.Delay(10000, stoppingToken);
            }
        }
    }
}

Program.cs
using Microsoft.EntityFrameworkCore;
using OrderApi.Data;
using OrderApi.Repositories;
using OrderApi.Mapping;
using OrderApi.Services;
using OrderApi.Middleware;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseInMemoryDatabase("OrderDb"));
```

```

builder.Services.AddScoped<IOrderRepository, OrderRepository>();

builder.Services.AddAutoMapper(typeof(MappingProfile));

builder.Services.AddHostedService<OrderBackgroundService>();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseMiddleware<ExceptionMiddleware>();

app.UseSwagger();
app.UseSwaggerUI();

app.MapControllers();

```

```
app.Run();
```

Testing

POST Example

```

{
  "customerName": "Divya",
  "totalAmount": 60000
}

```

Output:

```

{
  "id": 1,
  "customerName": "Divya",
  "totalAmount": 60000,
  "status": "High Value Order"
}

```

C# Problems

Area & Perimeter of Rectangle

using System;

```
class Shape
```

```

{
    // Base class (can be empty or common properties)
}

```

```
class Rectangle : Shape
```

```

{
    private int length;
    private int width;

    // Constructor
    public Rectangle(int length, int width)
    {
        this.length = length;
        this.width = width;
    }

```

```
// Method to calculate area
```

```

public int CalculateArea()
{
    return length * width;
}

```

```

    }

    // Method to calculate perimeter
    public int CalculatePerimeter()
    {
        return 2 * (length + width);
    }
}

class Program
{
    static void Main()
    {
        // Input
        string[] input = Console.ReadLine().Split(' ');
        int length = int.Parse(input[0]);
        int width = int.Parse(input[1]);

        // Object creation
        Rectangle rect = new Rectangle(length, width);

        // Output
        Console.WriteLine("Area: " + rect.CalculateArea());
        Console.WriteLine("Perimeter: " + rect.CalculatePerimeter());
    }
}

```

Supermarket Store

```

using System;
using System.Collections.Generic;

// Base Class
class Product
{
    public string Name;
    public double Price;
    public int Quantity;

    public Product(string name, double price, int quantity)
    {
        Name = name;
        Price = price;
        Quantity = quantity;
    }

    public virtual string GetProductDetails()
    {
        return $"{Name} - Price: {Price}, Quantity: {Quantity}";
    }
}

// Derived Class - Electronics
class Electronics : Product
{
    public int Warranty;

    public Electronics(string name, double price, int quantity, int warranty)
        : base(name, price, quantity)
    {
    }
}

```

```

    {
        Warranty = warranty;
    }

    public override string GetProductDetails()
    {
        return $"{Name} - Price: {Price}, Quantity: {Quantity}, Warranty: {Warranty} months";
    }
}

```

```

// Derived Class - Clothing
class Clothing : Product
{
    public string Size;

    public Clothing(string name, double price, int quantity, string size)
        : base(name, price, quantity)
    {
        Size = size;
    }

    public override string GetProductDetails()
    {
        return $"{Name} - Price: {Price}, Quantity: {Quantity}, Size: {Size}";
    }
}

```

```

// Inventory Class
class Inventory
{
    private List<Product> products = new List<Product>();

    public void AddProduct(Product product)
    {
        products.Add(product);
        Console.WriteLine($"Product added to inventory: {product.Name}");
    }

    public void DisplayInventory()
    {
        Console.WriteLine("\nInventory:");
        foreach (var p in products)
        {
            Console.WriteLine(p.GetProductDetails());
        }
    }

    public void CalculateTotalValue()
    {
        double total = 0;

        foreach (var p in products)
        {
            total += p.Price * p.Quantity;
        }

        Console.WriteLine($"Total value of the inventory: {total:F2}");
    }
}

```

```

}

// Main Program
class Program
{
    static void Main()
    {
        int num = int.Parse(Console.ReadLine());

        Inventory inventory = new Inventory();

        for (int i = 0; i < num; i++)
        {
            string input = Console.ReadLine();
            string[] parts = input.Split(',');

            string type = parts[0].Trim();
            string name = parts[1].Trim();
            double price = double.Parse(parts[2]);
            int quantity = int.Parse(parts[3]);

            if (type == "Electronics")
            {
                int warranty = int.Parse(parts[4]);
                inventory.AddProduct(new Electronics(name, price, quantity, warranty));
            }
            else if (type == "Clothing")
            {
                string size = parts[4].Trim();
                inventory.AddProduct(new Clothing(name, price, quantity, size));
            }
        }

        inventory.DisplayInventory();
        inventory.CalculateTotalValue();
    }
}
*****

```

Virus Resolving

```

class Program
{
    static void Main()
    {
        // Input
        string[] first = Console.ReadLine().Split(' ');
        int N = int.Parse(first[0]);
        int X = int.Parse(first[1]);
        int Y = int.Parse(first[2]);

        Console.ReadLine();

        int result;

        if (X < 2 * Y)
        {
            result = (N / 2) * X + (N % 2) * Y;
        }
        else

```

```

    {
        result = N * Y;
    }

    Console.WriteLine(result);
}
}
*****
Conditional View Rendering
Model/ScoreMode.cs
namespace MvcApp.Models
{
    public class ScoreModel
    {
        public string Name { get; set; } = string.Empty;
        public int Marks { get; set; }
    }
}
Controller/ScoreController.cs
using Microsoft.AspNetCore.Mvc;
using MvcApp.Models;

namespace MvcApp.Controllers
{
    public class ScoreController : Controller
    {
        // GET: Show form
        public IActionResult Index()
        {
            return View();
        }

        // POST: Handle input
        [HttpPost]
        public IActionResult Index(ScoreModel model)
        {
            return View(model);
        }
    }
}
Views/Score/Index.cshtml
@model MvcApp.Models.ScoreModel

<h2>Enter Student Details</h2>

<form asp-action="Index" method="post">
    <div>
        <label>Name</label>
        <input asp-for="Name" />
    </div>

    <div>
        <label>Marks</label>
        <input asp-for="Marks" />
    </div>

    <button type="submit">Check Result</button>
</form>

```

```

@if (Model != null)
{
    <h3>Result for @Model.Name</h3>

    @if (Model.Marks >= 50)
    {
        <p style="color:green">Pass</p>
    }
    else
    {
        <p style="color:red">Fail</p>
    }
}

```

Custom Middleware for Request Timing
 Middleware/RequestTimingMiddleware.cs
 using System.Diagnostics;

```

namespace MyApp.Middleware
{
    public class RequestTimingMiddleware
    {
        private readonly RequestDelegate _next;

        public RequestTimingMiddleware(RequestDelegate next)
        {
            _next = next;
        }

        public async Task Invoke(HttpContext context)
        {
            // Start time
            var stopwatch = Stopwatch.StartNew();

            // Call next middleware
            await _next(context);

            // Stop time
            stopwatch.Stop();

            // Calculate total time
            var timeTaken = stopwatch.ElapsedMilliseconds;

            // Add custom header
            context.Response.Headers["X-Response-Time"] = $"{timeTaken}ms";
        }
    }
}

```

Controllers/TestController.cs
 using Microsoft.AspNetCore.Mvc;

```

namespace RequestTimingApp.Controllers
{
    [ApiController]
    [Route("api/test")]
    public class TestController : ControllerBase
    {

```

```

    [HttpGet]
    public IActionResult Get()
    {
        // Simulate delay
        Thread.Sleep(200);

        return Ok("Request processed successfully");
    }
}
}
Program.cs
using RequestTimingApp.Middleware;

var builder = WebApplication.CreateBuilder(args);

// Add services
builder.Services.AddControllers();

var app = builder.Build();

// Use custom middleware
app.UseMiddleware<RequestTimingMiddleware>();

app.MapControllers();

app.Run();
*****
LINQ Projection Instead of Full Data
Model/Employee
namespace EmployeeApp.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public double Salary { get; set; }
        public string Department { get; set; } = string.Empty;
    }
}
Data/AppDbContext.cs
using Microsoft.EntityFrameworkCore;
using EmployeeApp.Models;

namespace EmployeeApp.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees => Set<Employee>();
    }
}
Controllers/EmployeeController.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

```

```

using EmployeeApp.Data;

namespace EmployeeApp.Controllers
{
    [ApiController]
    [Route("api/employees")]
    public class EmployeeController : ControllerBase
    {
        private readonly AppDbContext _context;

        public EmployeeController(AppDbContext context)
        {
            _context = context;
        }

        [HttpGet]
        public async Task<IActionResult> GetEmployees()
        {
            // LINQ Projection (IMPORTANT 🔥 )
            var result = await _context.Employees
                .Select(e => new
                {
                    e.Name,
                    e.Department
                })
                .ToListAsync();

            return Ok(result);
        }
    }
}

```

4. API with Dynamic Filtering (Query Parameters)

Models/Product.cs

namespace ProductFilterApp.Models

```

{
    public class Product
    {
        public string Name { get; set; } = string.Empty;
        public double Price { get; set; }
    }
}

```

Controllers/ProductController.cs

using Microsoft.AspNetCore.Mvc;

using ProductFilterApp.Models;

namespace ProductFilterApp.Controllers

```

{
    [ApiController]
    [Route("api/products")]
    public class ProductController : ControllerBase
    {
        // Dummy data (for testing)
        private static List<Product> products = new List<Product>
        {
            new Product { Name = "Laptop", Price = 70000 },
            new Product { Name = "Mouse", Price = 500 },
            new Product { Name = "Keyboard", Price = 1500 },
        }
    }
}

```

```

        new Product { Name = "Monitor", Price = 12000 }
    };

    [HttpGet]
    public IActionResult GetProducts(double? minPrice, double? maxPrice)
    {
        var query = products.AsQueryable();

        // Apply filtering only if parameters exist
        if (minPrice.HasValue)
        {
            query = query.Where(p => p.Price >= minPrice.Value);
        }

        if (maxPrice.HasValue)
        {
            query = query.Where(p => p.Price <= maxPrice.Value);
        }

        return Ok(query.ToList());
    }
}

Request GET /api/products=>Returns all products
GET /api/products?minPrice=1000=>Returns products ≥ 1000
GET /api/products?minPrice=1000&maxPrice=20000=>Returns products between range
*****

```

Cookie-Based Visit Counter (State Management)

Controllers/VisitController.cs

using Microsoft.AspNetCore.Mvc;

```

namespace CookieApp.Controllers
{
    public class VisitController : Controller
    {
        public IActionResult Index()
        {
            int visitCount = 0;

            // Check if cookie exists
            if (Request.Cookies.ContainsKey("VisitCount"))
            {
                // Read cookie value
                visitCount = int.Parse(Request.Cookies["VisitCount"]);

                // Increment count
                visitCount++;
            }
            else
            {
                // First visit
                visitCount = 1;
            }

            // Store updated value in cookie
            Response.Cookies.Append("VisitCount", visitCount.ToString());

            // Pass to view

```

```

        ViewBag.Count = visitCount;

        return View();
    }
}
Views/Visit/Index.cshtml
<h2>Visit Counter</h2>

<p>
    You have visited this page
    <strong>@ViewBag.Count</strong> times
</p>
ReadingCookie:
Request.Cookies["VisitCount"]=>This retrieves current visit count stored in the browser.
Writing Cookie:
Response.Cookies.Append("VisitCount", visitCount.ToString());=>This saves the incremented visit count back to the browser.
Output:
You have visited this page 5 times
*****

```

Part2:

Smart Form with Hidden Field Binding Issue

Models/OrderForm.cs

using System.ComponentModel.DataAnnotations;

namespace SmartFormApp.Models

```

{
    public class OrderForm
    {
        public string ProductName { get; set; } = string.Empty;

        public int Quantity { get; set; }

        // Optional field (IMPORTANT 🔥 )
        public string? DiscountCode { get; set; }
    }
}

```

Controllers/OrderController.cs

using Microsoft.AspNetCore.Mvc;

using SmartFormApp.Models;

namespace SmartFormApp.Controllers

```

{
    public class OrderController : Controller
    {
        // GET: Show form
        public IActionResult Create()
        {
            return View();
        }

        // POST: Handle form
        [HttpPost]
        public IActionResult Create(OrderForm model)
        {
            // ⚠ Important Fix (VERY IMPORTANT 🔥 )
            if (model.Quantity < 5)
            {

```

```

        // Ignore discount when not applicable
        model.DiscountCode = null;

        // Remove validation for hidden field
        ModelState.Remove("DiscountCode");
    }

    if (ModelState.IsValid)
    {
        ViewBag.Message = "Order Submitted Successfully";
        return View("Success", model);
    }

    return View(model);
}
}
}

```

Views/Order/Create.cshtml

@model SmartFormApp.Models.OrderForm

<h2>Create Order</h2>

<form asp-action="Create" method="post">

```

    <div>
        <label>Product Name</label>
        <input asp-for="ProductName" />
    </div>

```

```

    <div>
        <label>Quantity</label>
        <input asp-for="Quantity" id="qty" />
    </div>

```

```

<!-- Conditional Rendering -->
@if (Model != null && Model.Quantity >= 5)
{
    <div>
        <label>Discount Code</label>
        <input asp-for="DiscountCode" />
    </div>
}

```

```

    <button type="submit">Submit</button>
</form>

```

o **preserve value even when hidden used this hidden input**

```

@if (Model != null && Model.Quantity < 5)
{
    <input type="hidden" asp-for="DiscountCode" />
}

```

Views/Order/Success.cshtml

@model SmartFormApp.Models.OrderForm

<h2>Order Success</h2>

<p>Product: @Model.ProductName</p>

<p>Quantity: @Model.Quantity</p>

<p>Discount: @Model.DiscountCode</p>

Middleware that Modifies Response Body

Middleware/ResponseModificationMiddleware.cs

using System.Text;

namespace MiddlewareApp.Middleware

```
{
    public class ResponseModificationMiddleware
    {
        private readonly RequestDelegate _next;

        public ResponseModificationMiddleware(RequestDelegate next)
        {
            _next = next;
        }

        public async Task Invoke(HttpContext context)
        {
            // Step 1: Store original response stream
            var originalBodyStream = context.Response.Body;

            // Step2: Create a temporary memory stream
            using var memoryStream = new MemoryStream();
            context.Response.Body = memoryStream;

            // Step 3: Call next middleware (controller executes here)
            await _next(context);

            // Step 4: Read response from memory stream
            memoryStream.Seek(0, SeekOrigin.Begin);
            var responseText = await new StreamReader(memoryStream).ReadToEndAsync();

            // Step 5: Modify response content
            if (responseText.Contains("Hello"))
            {
                responseText = responseText.Replace(
                    "Hello",
                    "Hello - Processed by Middleware"
                );
            }

            // Step 6: Convert modified text to bytes
            var modifiedBytes = Encoding.UTF8.GetBytes(responseText);

            // Step 7: Restore original stream
            context.Response.Body = originalBodyStream;

            // Step 8: Write modified response back
            await context.Response.Body.WriteAsync(modifiedBytes, 0, modifiedBytes.Length);
        }
    }
}
```

Program.cs

using MiddlewareApp.Middleware;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

```

var app = builder.Build();

// Add custom middleware
app.UseMiddleware<ResponseModificationMiddleware>();

app.MapControllers();

app.Run();
Controllers/TestController.cs
using Microsoft.AspNetCore.Mvc;

namespace MiddlewareApp.Controllers
{
    [ApiController]
    [Route("api/test")]
    public class TestController : ControllerBase
    {
        [HttpGet]
        public IActionResult Get()
        {
            return Ok("Hello");
        }
    }
}
*****
Dynamic API Filtering with Multiple Conditions (Single Query Execution)
Models/Product.cs
namespace FilterApi.Models
{
    public class Product
    {
        public string Name { get; set; } = string.Empty;
        public double Price { get; set; }
        public string Category { get; set; } = string.Empty;
    }
}
Controllers/ProductController.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using FilterApi.Data;

namespace FilterApi.Controllers
{
    [ApiController]
    [Route("api/products")]
    public class ProductController : ControllerBase
    {
        private readonly AppDbContext _context;

        public ProductController(AppDbContext context)
        {
            _context = context;
        }

        [HttpGet]
        public async Task<IActionResult> GetProducts(
            double? minPrice,

```

```

    string? category,
    string? search)
{
    // Step 1: Start query
    var query = _context.Products.AsQueryable();

    // Apply filter ONLY if minPrice is provided
    if (minPrice.HasValue)
    {
        query = query.Where(p => p.Price >= minPrice.Value);
    }

    // Apply filter ONLY if category is provided
    if (!string.IsNullOrEmpty(category))
    {
        query = query.Where(p => p.Category == category);
    }

    // Apply filter ONLY if search is provided
    if (!string.IsNullOrEmpty(search))
    {
        query = query.Where(p => p.Name.Contains(search));
    }

    // Final Execution
    // .ToListAsync() is called ONLY at the end
    var result = await query.ToListAsync();

    return Ok(result);
}
}
}
GET /api/products?minPrice=100&category=Electronics&search=phone
Filters applied:
Price ≥ 100
Category = Electronics
Name contains "phone"

```

